Locksmith Lab

Realisation

Nikola Panayotov

Table of Contents

1 Introduction 2
2 Tools and Technologies Used 2
2.1 How to build and run 2
3 Code Structure and Design Alignment 2
3.1 Project file structure 3
3.2 Selected code excerpts 3
4 Building the Application 4
4.1 Recorded build step 4
4.2 Further implementation, read from the finished code 5
5 Demonstrated Weaknesses in the Unsecured Version 5
6 Defensive Measures in the Secured Version 6
7 Comparison: Unsecured vs Secured 7
8 Conclusion 7
References 8

1 Introduction

This document is the Realise evidence for the Security Playground project. It describes how SecureNotes was implemented as an Unsecured and a Secured version, based on a review of the actual source code, configuration and a local run of both APIs against a PostgreSQL 17 database, together with the one build step recorded in the realisation log (Section 4.1). The repository is available on GitHub at https://github.com/N1ki2K/Locksmith-Lab.

2 Tools and Technologies Used

Tool / technology	Version (as installed)	Role
Node.js + TypeScript	Node 22, TypeScript ~6.0 (frontend) / tsx runtime (API)	Implementation language, as decided in the Advice and Design.
React + Vite	React 19.2, Vite 8	Frontend framework and build/dev server.
Express	Express 5	REST API framework for both versions.
PostgreSQL	PostgreSQL 17 (17 in Unsecured docker-compose, 17-alpine in Secured)	Stores users and notes.
Docker / Docker Compose	docker-compose.yml per version	Runs the database as a reproducible container.
Argon2	argon2 (argon2id, memoryCost 19456, timeCost 2)	Password hashing in the Secured version only.
Helmet, express-rate-limit, connect-pg-simple	As declared in secured/api/package.json	Security headers, login rate limiting and server-side session storage in the Secured version.
Vitest + Supertest	secured/api/test/security.test.ts	Automated security tests for the Secured API.

2.1 How to build and run

Each version is started the same way, with its own database, port and environment file.

```
cd unsecured && docker compose up -d     # PostgreSQL 17 on port 5432
cd unsecured/api && npm install && npm start   # API on port 3000
cd unsecured/frontend && npm install && npm run dev   # Vite dev server on port 5173

cd secured && docker compose up -d       # PostgreSQL 17-alpine on port 5433
cd secured/api && npm install && npm start      # API on port 3001
cd secured/frontend && npm install && npm run dev   # Vite dev server on port 5174
```

Source code and full project: https://github.com/N1ki2K/Locksmith-Lab

3 Code Structure and Design Alignment

The table below checks the realised modules against the layered backend (routes, middleware, controllers, services, repositories) described in the Design.

Layer	Matches design?	Notes
Routes	Yes	authRoutes.ts and noteRoutes.ts map each endpoint to a controller, as planned. noteRoutes.ts attaches the authenticate middleware in both versions; the Secured version also attaches requireCsrf globally in index.ts.
Controllers	Yes, extended in Secured	Both read the request and build the HTTP response. The Secured authController and noteController add input validation (validUsername, validPassword, validNoteBody, parseId) that the Unsecured controllers do not have.
Services	Yes	authService and noteService implement authentication and note operations in both versions, matching the Design's separation of concerns.
Repositories	Yes, extended in Secured	Both execute SQL against PostgreSQL through parameterised queries using the pg driver. The Secured repositories additionally take the session user_id as a parameter on every note query, which the Unsecured repositories do not.
Middleware	Extended in Secured	authMiddleware.ts exists in both. The Secured version adds csrfMiddleware.ts, Helmet and express-rate-limit, none of which were present as separate modules in the Unsecured version.

3.1 Project file structure

The tree below shows the layout actually committed to the repository. Unsecured and Secured are two parallel copies of the same layered structure, each with its own frontend, api and database folder.

```
Locksmith-Lab/
├── unsecured/
│   ├── docker-compose.yml        PostgreSQL 17, port 5432
│   ├── database/init.sql         users, notes tables
│   ├── api/src/
│   │   ├── index.ts              Express app, session config, CORS
│   │   ├── database.ts           pg Pool
│   │   ├── routes/                authRoutes.ts, noteRoutes.ts
│   │   ├── middleware/authMiddleware.ts
│   │   ├── controllers/           authController.ts, noteController.ts
│   │   ├── services/              authService.ts, noteService.ts
│   │   └── repositories/          userRepository.ts, noteRepository.ts
│   └── frontend/src/
│       ├── App.tsx, main.tsx
│       ├── api/client.ts         fetch wrapper, credentials: include
│       └── pages/                 LoginPage, RegisterPage, NotesPage
├── secured/
│   ├── docker-compose.yml        PostgreSQL 17-alpine, port 5433
│   ├── database/init.sql         users, notes, secured_sessions tables
│   ├── api/src/                   same layers as Unsecured, plus:
│   │   ├── middleware/csrfMiddleware.ts
│   │   └── types/session.d.ts
│   ├── api/test/security.test.ts Vitest + Supertest security tests
│   └── frontend/src/              same pages, CSRF-aware api/client.ts
└── docs/                          Project plan, Analysis, Advice, Design,
                                   Realisation log, this document
```

3.2 Selected code excerpts

The three excerpts below were chosen because each is where the Secured implementation diverges most clearly from the Unsecured one: the session and cookie configuration, the password handling, and the note-ownership check.

Unsecured session configuration (unsecured/api/src/index.ts) — a fixed, hard-coded secret and a cookie with both security flags turned off:

```
app.use(session({
  secret: "password123",
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: false, secure: false },
}));
```

Secured session configuration (secured/api/src/index.ts) — the secret is read from the environment and rejected if weak, the cookie is scoped and locked down, and sessions are stored in PostgreSQL rather than only in memory:

```
if (!secret || secret.length < 32 || /^(password123|change[-_
]?me|secret|development)$/i.test(secret)) {
  throw new Error("SESSION_SECRET must be a unique secret of at least 32 characters");
}
...
app.use(session({
  name: "secured.sid",
  store: new PgSession({ pool: db, tableName: "secured_sessions", ... }),
  secret, resave: false, saveUninitialized: false,
  cookie: { httpOnly: true, secure: production, sameSite: "lax", maxAge: 1000 * 60 * 60 * 8,
path: "/" },
}));
```

Secured ownership check (secured/api/src/repositories/noteRepository.ts) — every note query is scoped to the session user, unlike the Unsecured equivalent which takes only the note id:

```
export async function findNoteById(id: number, userId: number) {
  const result = await db.query(
    "SELECT id, user_id, title, content, created_at, updated_at FROM notes WHERE id = $1 AND
user_id = $2",
    [id, userId],
  );
  return result.rows[0];
}
```

4 Building the Application

4.1 Recorded build step

The realisation log records one detailed step: the initial Express and TypeScript backend was created and connected to PostgreSQL running in Docker, using docker-compose.yml to start a postgres:17 container (Figure 1). A GET /api/health endpoint was added that runs a simple database query to confirm connectivity. After resolving environment-variable and Docker configuration issues, the endpoint returned {"status":"ok","database":"connected"} (Figure 2), which is the same response format present in both versions today.

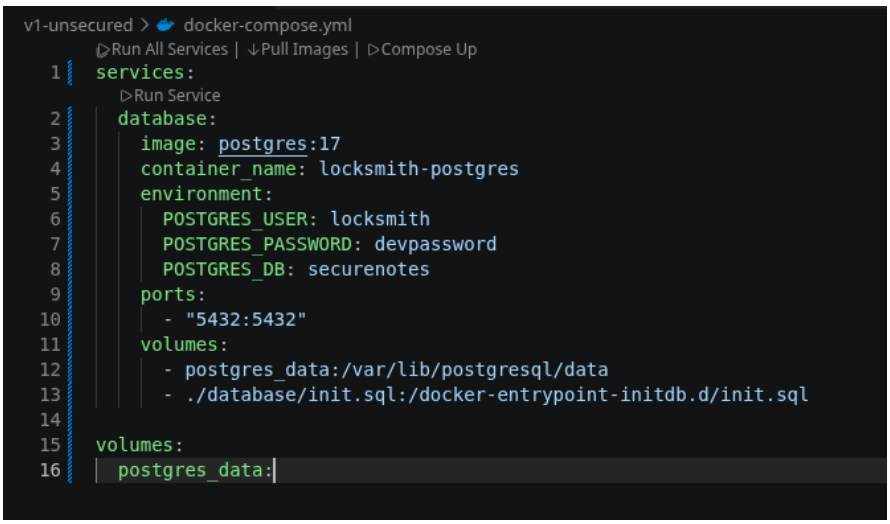


Figure 1. docker-compose.yml for the Unsecured PostgreSQL container, from the realisation log.

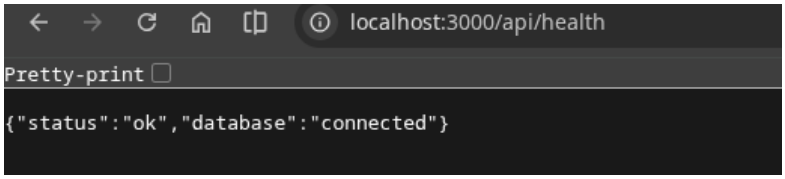


Figure 2. GET /api/health returning status: ok and database: connected, from the realisation log.

4.2 Further implementation, read from the finished code

The realisation log does not describe every later step individually. The remaining build order below is reconstructed from the finished repository rather than from a day-by-day diary: the authentication routes and session handling were added, followed by the note CRUD routes and repositories, then the two React frontends, and finally the Secured-only layer of input validation, Argon2 password hashing, CSRF protection, Helmet headers, rate limiting and PostgreSQL-backed sessions. This order matches the layered flow set out in the Design (routes → middleware → controllers → services → repositories) and the fact that the Secured version reuses the Unsecured page components with a CSRF-aware API client.

5 Demonstrated Weaknesses in the Unsecured Version

The findings below were reproduced by running the Unsecured API locally against PostgreSQL 17 and sending real requests with curl, not inferred from the code alone.

Weakness	Evidence	Mapped OWASP Top 10 area
Plaintext password storage	Registering alice with password alicepw123 and querying the users table returned password = alicepw123 in plain text.	Cryptographic Failures
Broken access control between users (IDOR)	User bob created a note (id 1). User alice, logged in as herself, retrieved GET /api/notes/1 and received Bob's full note content.	Broken Access Control
No CSRF protection	Every state-changing request (register, login, create/update/delete note) succeeded with only the session cookie attached; no token of any kind was required or checked.	Broken Access Control
Insecure cookie configuration	The session cookie set by the Unsecured API (connect.sid) has no HttpOnly flag and no Secure flag, confirmed by inspecting the cookie jar after login.	Security Misconfiguration

Weakness	Evidence	Mapped OWASP Top 10 area
Hard-coded session secret	The Express session secret is the literal string "password123" in unsecured/api/src/index.ts.	Cryptographic Failures
No input validation or rate limiting	Arbitrarily long or malformed note titles and content, and unlimited login attempts with wrong passwords, were all accepted without any 400 or 429 response.	Security Misconfiguration

Cross-site scripting was planned as a test area, but could not be demonstrated against either version: both frontends render note content through React, which escapes text content by default, and no dangerouslySetInnerHTML or equivalent HTML sink exists in unsecured/frontend/src or secured/frontend/src. A stored <script> payload was saved and returned by the API unchanged, as plain text, in both versions, but it does not execute in the browser.

6 Defensive Measures in the Secured Version

The same requests were repeated against the Secured API. Each defence below was triggered and observed directly, not only read from the source.

Area	Verified behaviour	Evidence
Password storage	Registering alice through the Secured API stored an Argon2id hash ($argon2id$v=19$m=19456,t=2,p=1$...), never the plain password.	Database query on secured users table
CSRF protection	A register or login request sent without an X-CSRF-Token header, or with the wrong token, was rejected with 403 {"error":"CSRF token required"} or "Invalid CSRF token".	curl without / with header
Session cookie flags	The Secured cookie, secured.sid, is written to the cookie jar with the HttpOnly_ prefix curl uses for HttpOnly cookies, and carried Secure/SameSite=Lax attributes in the response headers.	Cookie jar + response headers
Broken access control fixed	Alice, authenticated but not owning Bob's note, requested GET /api/notes/1 and received 404 Note not found, instead of the note contents returned by the Unsecured version.	curl with alice's session
Login rate limiting	Eleven consecutive wrong-password login attempts on one session returned 401 for the first attempts and then 429 {"error":"Too many login attempts; try again later"}.	curl loop
Input validation	A 4,100-character note, a note with an extra user_id field, and a non-numeric note id were all rejected with a 400 error naming the limit or the problem.	curl with oversized/invalid bodies
Security headers	Responses carried Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN, none of which are present on Unsecured responses.	Response headers from Helmet

Area	Verified behaviour	Evidence
Session invalidation on logout	After POST /api/auth/logout, a further request with the same cookie returned 401, confirming the server-side session was destroyed rather than only the client cookie cleared.	curl after logout
Automated regression tests	secured/api/test/security.test.ts (Vitest + Supertest) checks that CSRF is required, that SQL-like strings are stored as data, that login rotates the session id, and that each user can only operate on owned notes.	Test file in the repository

7 Comparison: Unsecured vs Secured

Aspect	Unsecured	Secured
Password storage	Plain text	Argon2id hash
Session secret	Hard-coded "password123"	Required from environment, rejected if weak or default
Session storage	In-memory only	PostgreSQL-backed (connect-pg-simple)
Cookie flags	httpOnly: false, secure: false	httpOnly: true, secure in production, sameSite: lax
CSRF protection	None	Required token on every state-changing request
Note ownership check	Not enforced (IDOR possible)	Enforced in every repository query
Rate limiting	None	10 login attempts per 15 minutes
Input validation	None beyond basic type use	Username, password, note length and field checks
Security headers	None added	Helmet: CSP, HSTS, X-Content-Type-Options, X-Frame-Options
Automated tests	None	security.test.ts (Vitest + Supertest)

8 Conclusion

SecureNotes was realised as two parallel Express and PostgreSQL applications sharing the same layered design and the same React pages. The Unsecured version reproduces, with evidence from actual requests rather than inspection alone, plaintext password storage, a cross-user IDOR, the absence of CSRF protection, an insecure session cookie and a hard-coded session secret. The Secured version addresses each of these with Argon2id hashing, a required and validated session secret, PostgreSQL-backed sessions, locked-down cookie flags, CSRF tokens, per-user ownership checks, login rate limiting, input validation and Helmet security headers, backed by an automated test suite. Cross-site scripting could not be demonstrated in either version, because both render note content through React's default text escaping and neither contains an HTML injection sink

.

References

Express. (n.d.). *Express - Node.js web application framework*. https://expressjs.com/

Helmet. (n.d.). *Helmet.js documentation*. https://helmetjs.github.io/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

React. (n.d.). *React*. https://react.dev/

Simov, D. (n.d.). *Argon2 for Node.js*. https://www.npmjs.com/package/argon2

Vitest. (n.d.). *Vitest documentation*. https://vitest.dev/